

งาน: พัฒนาระบบแนะนำเป็นระบบของตัวเอง

ระบบแนะนำหนังสือ

BOOK RECOMMENDER SYSTEM · Neo4j (Graph DB) + Streamlit

ข้อมูลของเราเอง 12 คน × 28 เล่ม × 51 ความชอบ

ผลการแนะนำแสดง “ภาพปกหนังสือจริง” ทุกครั้ง

มีทั้งโน้ตบุ๊กอธิบายวิธีทำ (Colab), เว็บแอปสาธิต และสไลด์ชุดนี้บน GitHub

12

ผู้ใช้ในระบบ

28

หนังสือ (14 หมวด)

51

ความชอบ

3+1

วิธีให้คะแนน

จัดทำโดย: รหัส 007 · วิชา _____ · วันที่ ____/____/____

AGENDA

หัวข้อที่จะนำเสนอ

01

โจทย์และสิ่งที่ส่ง

งาน 4 ข้อของอาจารย์ → ส่งครบอะไรบ้าง

02

แนวคิดระบบแนะนำกราฟ

ทำไมข้อมูล “ใครชอบอะไร” ต้องใช้กราฟ + การเดิน 3 hop

03

ข้อมูลและสถาปัตยกรรม

ข้อมูลชุดของเรา + Neo4j + Streamlit + GitHub

04

ระบบทำงานจริง

ผลการแนะนำ 3 วิธี + ภาพปก + เหตุผลย้อนหลังได้

05

สถิติการใช้งาน (เดโมสด)

เพิ่มความชอบแล้วคำแนะนำเปลี่ยนทันทีโดยไม่ต้องเทรนใหม่

06

การทดสอบและลิงก์งาน

ผลตรวจสอบจริง + Colab + GitHub + หน้า index

งานที่อาจารย์สั่ง 4 ข้อ — ส่งครบทุกข้อ

1

Present ด้วย **PowerPoint** + สาธิตการใช้งานระบบ

สไลด์ 16 หน้า (ไฟล์นี้) + เดโมสโตนแอป + ใส่ภาพในระบบแนะนำจริง (หน้าปกหนังสือ)

2

ข้อมูล **PowerPoint** ไว้ใน **GitHub**

ใส่ทั้ง .pptx และ .pdf ในโฟลเดอร์ slides/ ของ repo

3

เอาการบ้านทุกอันไว้ใน **GitHub**

ทุกชิ้นงานมี repo ของตัวเอง + โฟลเดอร์กลาง homework รวมลิงก์

4

จัดหน้า **index** ให้ลิงก์กับงานทุกอัน

หน้าเว็บ index.html บน GitHub Pages ลิงก์ทุก repo

ทำไมต้องใช้ฐานข้อมูลกราฟ?

ข้อมูล “ใครชอบหนังสือเล่มไหน” เป็นเรื่องของความสัมพันธ์ — ถ้าเก็บเป็นตาราง จะต้อง JOIN หลายชั้นกว่าจะได้คำตอบ แต่ในกราฟเขียนครั้งเดียวจบ

คำแนะนำทั้งระบบเขียนด้วย **Cypher** ไม่เก็บรหัส

```
MATCH (me:User {user: 'สมชาย'})-[:LIKES]->
      (shared:Book)<-[:LIKES]-(other:User)
MATCH (other)-[:LIKES]->(rec:Book)
WHERE NOT (me)-[:LIKES]->(rec)
RETURN rec.title, count(DISTINCT other) AS votes
ORDER BY votes DESC
```

ขั้นตอนการเดิน 3 hop

1. ดูหนังสือที่ผู้ใช้ชอบ
2. หาคำอื่นที่ชอบเล่มเดียวกัน = รสนิยมใกล้เคียงกัน
3. ดูเล่มอื่นที่คนกลุ่มนั้นชอบ (ที่เรายังไม่ชอบ)
4. นับ/ให้คะแนน → เรียงอันดับ = คำแนะนำ

ตัวอย่างจริง: สมชาย

ชอบอยู่แล้ว 4 เล่ม

Harry Potter · The Hobbit · LOTR · Da Vinci Code

คณรสนิยมใกล้เคียง (**Jaccard**)

นรินทร์ 0.500 (ชอบร่วม 3 เล่ม)

มะลิ 0.333 (ชอบร่วม 2 เล่ม)

พลอย 0.143 (ชอบร่วม 1 เล่ม)

ระบบแนะนำ

The Hunger Games — 0.833

Dune — 0.500

The Lightning Thief — 0.333

เพราะมะลิกับนรินทร์ต่างชอบ Hunger Games ทั้งคู่

12 คน × 28 เล่ม × 51 ความชอบ (เก็บเอง ไม่ใช่ dataset สำเร็จรูป)

12

ผู้ใช้ (เพื่อนในห้อง)

28

หนังสือ

14

หมวดหนังสือ

51

ความสัมพันธ์ LIKES

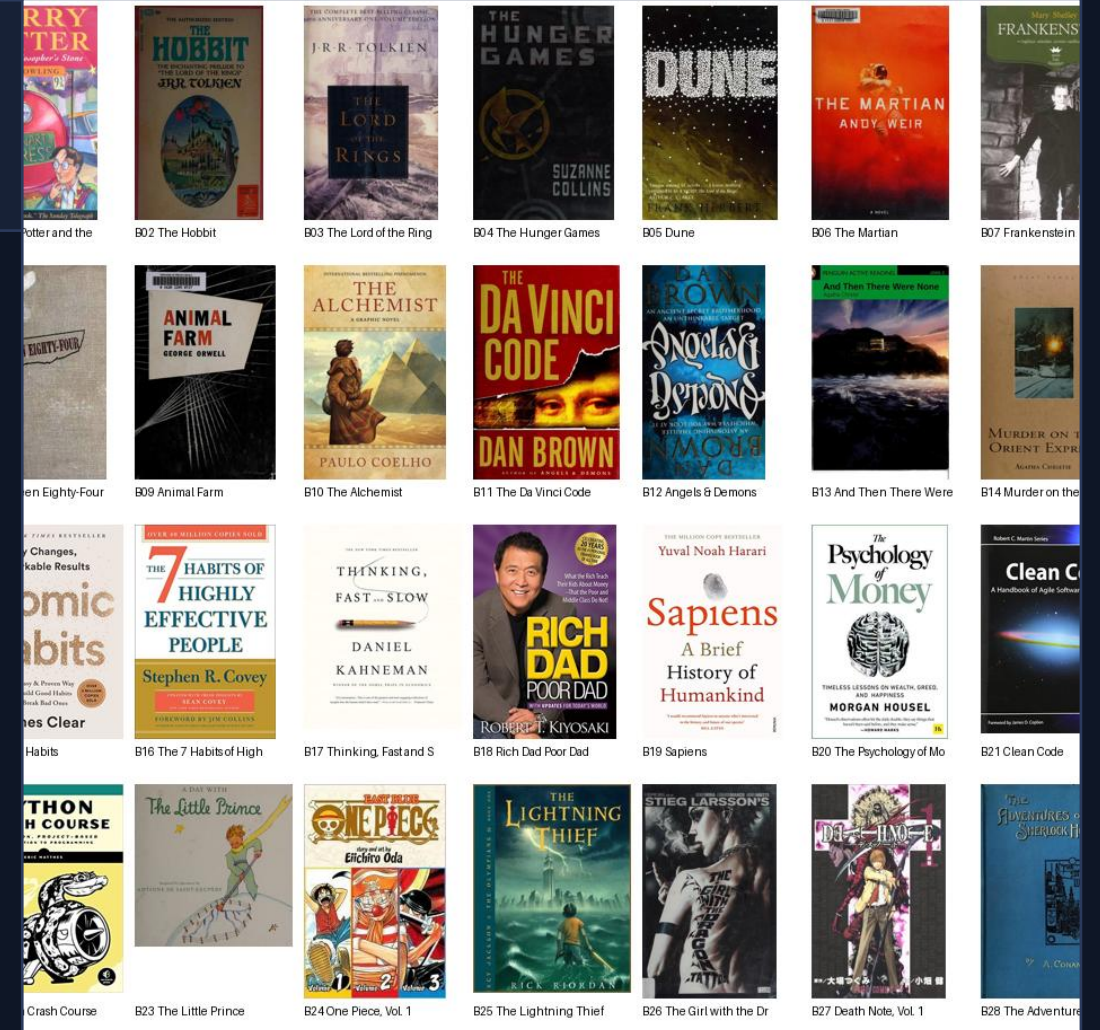
ปกหนังสือทุกเล่มดึงจาก Open Library (ฟรี ไม่ต้องใช้ API key) แล้วเก็บไฟล์ไว้ในโปรเจกต์

→ ภาพยังแสดงได้แม้ไม่มีอินเทอร์เน็ต และเปิดใน Colab ก็เห็นภาพ

หมวด: แฟนตาซี · ไซไฟ · สืบสวน · พัฒนาตัวเอง · การเงิน · การ์ตูน · คลาสสิก ฯลฯ

ขั้นตอนเตรียมข้อมูล

- ค้นหาแต่ละเล่มใน Open Library API → ได้ ISBN + รหัสภาพปก
- ดาวน์โหลดภาพปกเก็บเป็น asset ของ repo (ตรวจด้วยตาว่าตรงเรื่องทุกเล่ม)
- กำหนดความชอบของ 12 คนให้มิกค์มรสนิยมชัดเจน และไม่มีหนังสือเล่มไหนไร้เจ้าของ



ระบบประกอบด้วย 4 ส่วน และทุกส่วนอยู่บน **GitHub**

Colab Notebook

ทดลอง + อธิบายวิธีทำ

Cypher ทึละชั้น มีผลรันจริงฝังในไฟล์

Neo4j

ฐานข้อมูลกราฟ

12 User · 28 Book · 14 Genre ·
51 LIKES

Streamlit App

หน้าจอสาริต 5 แท็บ

การ์ดภาพปก · กราฟ · เดโมสด

GitHub + Pages

repo ระบบ + โฟลเดอร์ homework

หน้า index รวมลิงก์งานทุกชิ้น

การไหลของข้อมูล

- ① เตรียมข้อมูล — สคริปต์ดึงข้อมูล + ภาพปกจาก Open Library → data/books.json
- ② โหลดเข้า **Neo4j** — MERGE โหนด/ความสัมพันธ์ + สร้าง constraint กันข้อมูลซ้ำ
- ③ ให้บริการคำแนะนำ — Cypher 3 วิธี + วิธีผสม (ถ่วงน้ำหนัก Jaccard)
- ④ แสดงผล — Streamlit แสดงการ์ดภาพปก + กราฟ + เดโมสดแก้ข้อมูลในฐานข้อมูลจริง

สกีแมกราฟและคำสั่งที่ใช้จริง

(User) -[:LIKES]-> **(Book)** -[:IN_GENRE]-> **(Genre)**

บุคลิกของข้อมูล: ผู้ใช้ → ความชอบ → หนังสือ → หมวด

ข้อดีที่ได้จากโครงสร้างนี้

- ถ้ามจากโหนดไหนก็ได้ ไม่ต้อง JOIN หลายตาราง
- เพิ่มคน/หนังสือใหม่ = เพิ่มโหนด ไม่ต้องแก้โครงสร้าง
- โหนด Genre เป็นทั้ง "สัญญาณที่สอง" และตัวช่วยแก้ cold start

Cypher ตัวอย่าง

```
MATCH (u:User)-[:LIKES]->(b:Book)
RETURN u.user AS user, count(b) AS books_liked
ORDER BY books_liked DESC
```

// จำนวนจริงในฐานข้อมูล

// User 12 · Book 28 · Genre 14 · LIKES 51

กันข้อมูลซ้ำตอนโหลด (**constraint + MERGE**)

```
CREATE CONSTRAINT user_name IF NOT EXISTS
FOR (u:User) REQUIRE u.user IS UNIQUE;
```

```
UNWIND $rows AS row
MERGE (u:User {user: row.user})
SET u.age = row.age, u.group = row.group;
```

หน้าแนะนำหนังสือ — มีภาพปกในระบบแนะนำตามโจทย์

ฐานข้อมูล: Neo4j
(bolt://127.0.0.1:7687)

ผู้ใช้ในระบบ
12

หนังสือ 28 ความชอบ 51 หมวด 14

เลือกผู้ใช้ที่จะแนะนำหนังสือให้
สมาชิก

วิธีให้คะแนน

- ☒ ถ่วงน้ำหนัก (Jaccard)
- ☐ นับโหนดเพื่อน
- ☐ ตามหมวดหนังสือ
- ☐ ผสม: เพื่อน + หมวด

จำนวนที่แนะนำ
3

อัลกอริทึม: Collaborative Filtering บนกราฟ (เดิน 3 hop) + Content-based ผ่านโหนด Genre

คำแนะนำสำหรับ สมาชิก · วิธี: ถ่วงน้ำหนัก (Jaccard) · แสดง 3 เล่มจาก 28 เล่มในระบบ

แนะนำหนังสือ คลังหนังสือ โครงสร้างกราฟ เติบโต: เพิ่มความชอบ เกี่ยวกับระบบ

ชอบอยู่แล้ว หนังสือที่ระบบแนะนำ

หนังสือสิ่งอันดับของ สมาชิก — ใช้เป็นจุดเริ่มของกราฟ (4 เล่ม)



Harry Potter and the Philosopher's Stone
J. K. Rowling · แฟนตาซี



The Da Vinci Code
Dan Brown · สืบสวน



The Hobbit
J. R. R. Tolkien · แฟนตาซี



The Lord of the Rings
J. R. R. Tolkien · แฟนตาซี



The Hunger Games
Suzanne Collins · ยาวาน/ไซไฟ

★ คะแนน 0.833

เพื่อนที่ชอบ: มะลิ, นรินทร์



Dune
Frank Herbert · ไซไฟ

★ คะแนน 0.5

เพื่อนที่ชอบ: นรินทร์



The Lightning Thief
Rick Riordan · ยาวาน/แฟนตาซี

★ คะแนน 0.333

เพื่อนที่ชอบ: มะลิ

ใครมีรสนิยมใกล้เคียง สมาชิก (Jaccard similarity)

ผู้ใช้	ชอบร่วมกัน (เล่ม)	หนังสือที่ชอบรวม	Jaccard
นรินทร์		3 B01, B02, B03	0.5
มะลิ		2 B01, B02	0.333
พลอย		1 B11	0.143

การ์ดแนะนำ

ภาพปก + ชื่อเรื่อง + ผู้เขียน + หมวด

คะแนน

มาจากความคล้ายของคนที่ชอบเล่มนี้

เหตุผล

บอกชื่อเพื่อนและเล่มที่ชอบร่วมกัน

ข้อมูลตั้งต้น

แสดงหนังสือที่ผู้ใช้ชอบอยู่แล้ว

ระบบแนะนำหนังสือ · Neo4j + Streamlit · จัดทำโดย รหัส 007

8

ให้คะแนน 3 วิธี + วิธีผสม — ผลจริงของผู้ใช้ “สมชาย”

นับโหวต

1 คน = 1 เสียง

The Hunger Games 2 โหวต

Dune 1 โหวต

อื่น ๆ 1 โหวต

ปัญหาที่เจอ: เพื่อนทุกคนมีน้ำหนักเท่ากัน แม้รสนิยมจะใกล้เคียงไม่เท่ากัน

ถ่วงน้ำหนัก (Jaccard)

คะแนน = $\sum$ ความคล้าย

The Hunger Games 0.833

Dune 0.500

The Lightning Thief 0.333

แก้ปัญหาดังเดิม: คนรสนิยมใกล้เคียงมีน้ำหนักมากที่สุด

ตามหมวด + ผสม

เพื่อน 60% + หมวด 40%

The Hunger Games 0.600

And Then There Were None 0.503

Dune 0.400

ช่วยผู้ใช้ใหม่ที่ยังไม่มีความคล้ายกับใคร (cold start)

สูตรถ่วงน้ำหนัก: $Jaccard(เรา, เขา) = \frac{|\text{หนังสือที่ชอบร่วมกัน}|}{|\text{หนังสือของสองคนรวมกัน}|} \rightarrow$ ใช้เป็นน้ำหนักโหวตของแต่ละคน แล้วบวกกันเป็นคะแนนของเล่มนั้น

ทุกคำแนะนำอธิบายย้อนหลังได้: บอกได้ว่าเล่มนี้มาจากเพื่อนคนไหน และเพราะคุณชอบเล่มอะไร

เดโมส: เพิ่มความชอบ → คำแนะนำเปลี่ยนทันที (ไม่ต้องเทรนโมเดลใหม่)

คำแนะนำหนังสือ

Neo4j + Streamlit · รหัส 007

ข้อมูล: Neo4j

://127.0.0.1:7687)

ความชอบ

หมวด

51

14

คำแนะนำหนังสือให้

นัก (Jaccard)

เพื่อน

หนังสือ

อน + หมวด

ภาพ

3

Collaborative Filtering บน

hop) + Content-based ผ่าน

ระบบแนะนำหนังสือ

คำแนะนำสำหรับ สมาชิก · วิธี: ตัวงน้ำหนัก (Jaccard) · แสดง 3 เล่มจาก 28 เล่มในระบบ

แนะนำหนังสือ

คลังหนังสือ

โครงสร้างกราฟ

เดโมส: เพิ่มความชอบ

เกี่ยวกับระบบ

เดโมส: เพิ่มหนังสือที่ชอบ แล้วดูว่าคำแนะนำเปลี่ยนทันที

แก้ไขข้อมูลในฐานข้อมูลกราฟจริง แล้วคำนวณใหม่ทันที — ระบบไม่ต้องเทรนโมเดลใหม่

เลือกเล่มที่ สมาชิก เพิ่มว่าชอบ

B01 · Harry Potter and the Philosopher's Stone

ผลก่อน-หลังจะแสดงด้านล่าง

+ เพิ่มความชอบ

แก้ไขข้อมูลดั้งเดิมของ สมาชิก

Harry Potter and the Philosopher's Stone

ลบ

The Da Vinci Code

ลบ

The Hobbit

ลบ

The Lord of the Rings

ลบ

ระบบแนะนำหนังสือ · จัดทำโดย รหัส 007 · ข้อมูลและภาพปกจาก Open Library (ฟรี) · กราฟ: Neo4j · ส่วนติดต่อผู้ใช้: Streamlit

สถิติตามลำดับนี้

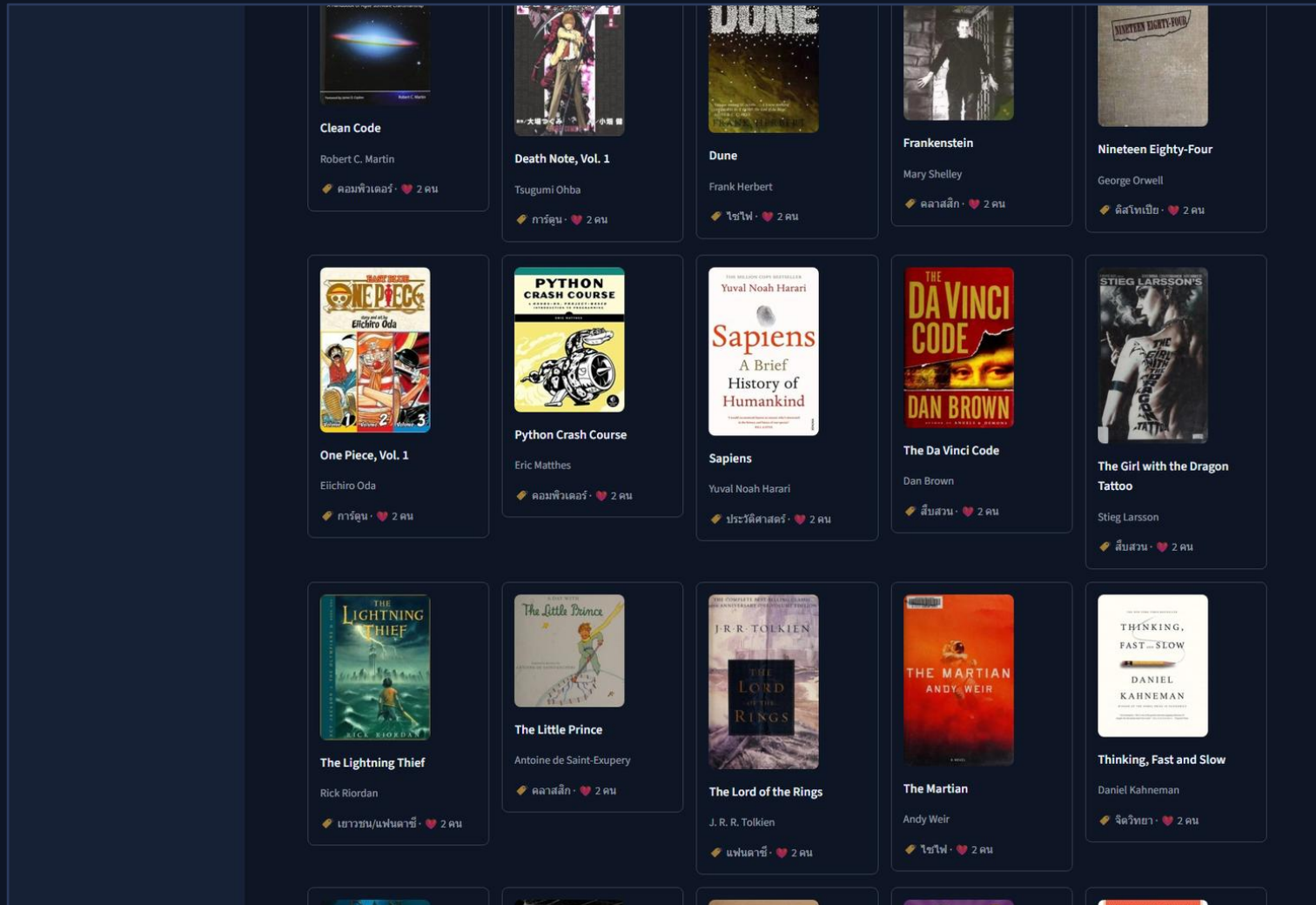
1. เปิดหน้า "เดโมส: เพิ่มความชอบ"
2. เลือกเล่มที่ต้องการเพิ่ม (เช่น The Martian)
3. กด "+" เพิ่มความชอบ" → เขียนลง Neo4j ทันที
4. ตารางเทียบ ก่อน/หลัง จะโชว์ว่าคำแนะนำเปลี่ยนเป็นเล่มใหม่
5. ลบข้อมูลที่เพิ่ม เพื่อคืนสภาพเดิม (กดปุ่มลบได้)
6. สลับวิธีให้คะแนนเพื่อโชว์ผลต่างของแต่ละวิธี

จุดที่ทำให้ระบบนี้ "เป็นระบบของตัวเอง": อัลกอริทึมของเรา + ข้อมูลของเรา + หน้าจอของเรา และเขียนคำสั่งกราฟเองทั้งหมด

ระบบแนะนำหนังสือ · Neo4j + Streamlit · จัดทำโดย รหัส 007

10

คลังหนังสือ 28 เล่ม — ค้นหาได้ และแสดงปกครบทุกเล่ม



สิ่งที่หน้าแสดง

- ค้นหาด้วยชื่อเรื่อง / นักเขียน / หมวด
- แต่ละการ์ด: ปก + ชื่อ + หมวด + จำนวนคนที่ชอบ
- ใช้ตรวจสอบว่าข้อมูลในกราฟครบ 28 เล่มจริง

เทคโนโลยีที่ใช้

Neo4j — ฐานข้อมูลกราฟ (Cypher)

Streamlit — ส่วนติดต่อผู้ใช้บนเว็บ (ฟรี)

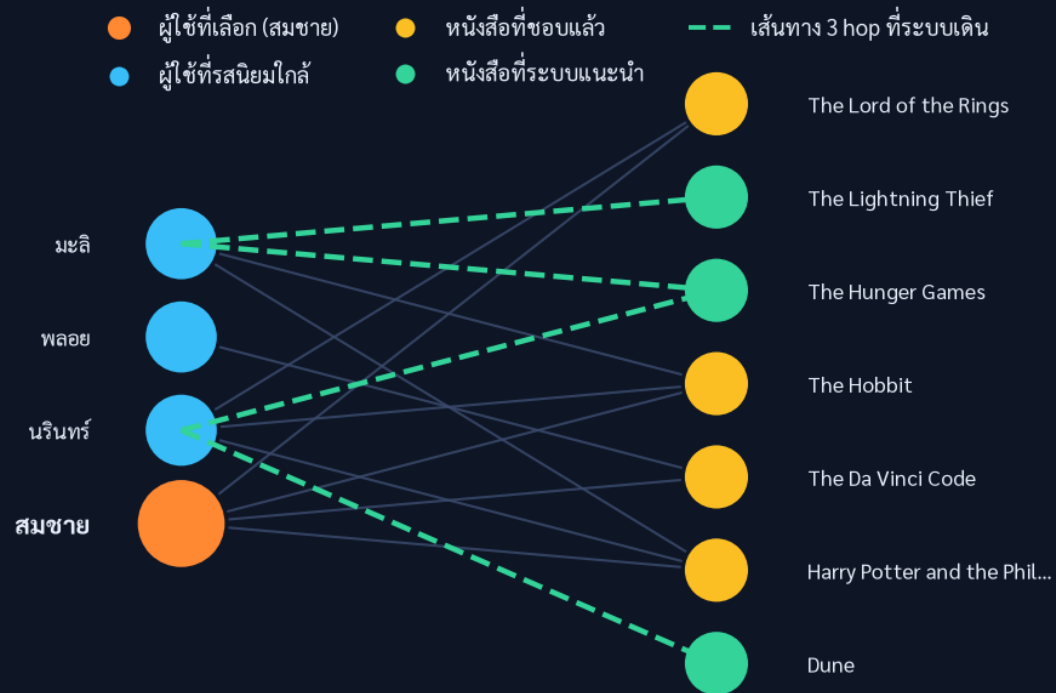
networkx + matplotlib — วาดกราฟความสัมพันธ์

Open Library API — ข้อมูลและภาพปก (ฟรี)

Colab + gist — โน้ตบุ๊กอธิบายวิธีทำ

ภาพกราฟความสัมพันธ์ที่ระบบวาดเอง (ผู้ใช้ ↔ หนังสือ)

กราฟคำแนะนำสำหรับ สมชาย



● ส้ม = ผู้ใช้ที่เลือก (สมชาย)

● ฟ้า = ผู้ใช้ที่ระบบแนะนำ

● เหลือง = หนังสือที่ชอบแล้ว

● เขียว = หนังสือที่แนะนำ

- - - เส้นประ = เส้นทาง 3 hop

Cold start และการเทียบกับ baseline ยอดนิยม

ผู้ใช้ใหม่ที่ชอบเล่มเดียว (ที่ยังไม่มีใครชอบ)

ผู้ใช้ใหม่ชอบ: **One-Punch Man, Vol. 1** (หมวดการ์ตูน)

ผลจากสัญญาณเพื่อน → ว่างเปล่า (cold start จริง)

ผลจากสัญญาณหมวดหนังสือ → **Death Note Vol. 1 · One Piece Vol. 1**

บทสรุป: ถ้าใช้เฉพาะ collaborative filtering ผู้ใช้แบบนี้จะไม่ได้คำแนะนำเลย แต่เพราะมีโหนด Genre ระบบจึงยังแนะนำได้ — และเมื่อผู้ใช้ใหม่มีความชอบเพิ่ม คำแนะนำจะเปลี่ยนไปตามเพื่อนที่รสนิยมตรงกัน

เทียบกับ **baseline ‘ยอดนิยม’**

แนะนำแบบไม่ดูว่าใครเป็นใคร

```
MATCH (b:Book) <-[:LIKES]-(u:User)
RETURN b.title, count(u) AS likes
ORDER BY likes DESC LIMIT 5
```

Harry Potter · The Hobbit · The Hunger Games (3 คน)

ทุกคนจะได้ลิस्टเดียวกันเสมอ

ระบบของเราดีกว่า เพราะดูความคล้ายของแต่ละคน — สมชายได้ Hunger Games แต่ ‘ธนกัร’ ได้ Dune/Frankenstein/The Martian และ ‘มินต์’ ได้สายพัฒนาตัวเอง

ผลรันจริงจากโน้ตบุ๊ก (ฝั่ง output ในไฟล์ส่ง): ทุกเซลล์รันผ่าน · การ์ดภาพปกปรากฏ 16 จุด · ฐานข้อมูลกลับสู่สภาพ 12/28/51 หลังทดลอง cold start

ผลตรวจสอบจริง (ไม่ใช่คำอธิบาย)

โหลดข้อมูลเข้า **Neo4j**

`tools/load_neo4j.py`

12 ผู้ใช้ · 28 หนังสือ · 51 LIKES · 14 Genre

ความสอดคล้อง **2 backend**

`tools/consistency_test.py`

Cypher (Neo4j) กับ backend สำรองให้ผลตรงกัน 12 คน × 5 การตรวจ

โน้ตบุ๊กทั้งเล่ม

`py -3.13 tools/verify_ipynb_cells.py`

VERDICT: PASS — ทุก code cell รันได้ (12 เซลล์)

ภาพปกครบถ้วน

`contact sheet` + ตรวจสอบด้วยสายตา

28/28 เล่มมีปกจริง (แค่ 2 เล่มที่เป็นภาพเปล่า)

หน้าจอแอปจริง

`tools/shots.py` (Playwright)

ถ่าย 6 ภาพ ทุกแท็บแสดงปกครบ ไม่มีภาพพัง

หมายเหตุ: ผลการรันในโน้ตบุ๊กมาจาก Neo4j ที่ทดสอบในเครื่อง (เซลล์เชื่อมต่อยังคงเป็นบัญชีของเรา พร้อมรหัสผ่านที่กรอกเองตอนรันบน Colab)

เปิดดูได้ทันที / รันเองได้ใน 3 คำสั่ง

โน้ตบุ๊กอธิบายวิธีทำ (Colab)

https://colab.research.google.com/gist/Nasak16/247492673b9b3a74dd39bd7d442d398c/BookRecommender_Neo4j_007.ipynb

โค้ดระบบทั้งหมด (GitHub)

<https://github.com/Nasak16/book-recommender>

หน้า **index** รวมงานทุกชิ้น

<https://nasak16.github.io/homework/>

รันในเครื่อง (3 คำสั่ง)

```
py -3.13 -m pip install -r requirements.txt
py -3.13 tools/load_neo4j.py <neo4j-uri> neo4j <password>
py -3.13 -m streamlit run app.py
```

ถ้าชื่อ Neo4j ไม่ได้ แอปจะสลับไปใช้ backend สำรองอัตโนมัติ และแจ้งเตือนบนแถบข้าง — หน้าจอและภาพปกยังทำงานครบถ้วน

รหัสผ่าน Neo4j ไม่ถูกอัปโหลด (อยู่ใน .gitignore) — ในโปรเจกต์มีแค่ไฟล์ตัวอย่าง secrets.toml.example

ระบบแนะนำของเรา + สิ่งที่อยู่ยอดได้

ได้อะไรจากงานนี้

- ระบบแนะนำที่ใช้ได้จริงบนฐานข้อมูลกราฟ + ข้อมูลของเราเอง
- แสดงภาพปฏิกิริยาในผลการแนะนำ (ตามโจทย์ข้อ 1)
- คำแนะนำอธิบายย้อนหลังได้ และทดสอบ cold start แล้ว
- มีทั้งโน้ตบุ๊ก เว็บแอปสโตร์ และหน้า index รวมงานบน GitHub

ข้อจำกัด และก้าวต่อไป

- ข้อมูล 51 ความชอบข่งน้อย (sparsity) — เก็บจากผู้ใช้งานจริงเพิ่มจะแน่นขึ้น
- ยังไม่มีคะแนนแบบ 1-5 ดาว จึงถ่วงน้ำหนักได้แต่ความคล้ายรสนิยม
- ต่อยอด: ใช้ tag ละเอียดยกกว่า 14 หมวด, เพิ่ม PageRank/community detection, ทำระบบสมัครสมาชิกจริงบน Streamlit Cloud

ขอบคุณครับ — ยินดีตอบคำถาม / สาริตศดได้ทันที